

Grid Search, Random Search, K-Fold Cross-Validation, dan Tracking Hasil

Taufiqurrahman

Tujuan Pembelajaran

- Memahami **Grid Search** dan **Random Search** untuk tuning hyperparameter.
- Menerapkan **K-Fold Cross-Validation** (termasuk variasi stratified dan time-aware).
- Menyusun **skema tracking hasil** eksperimen yang rapi dan reproduibel.

1 Dasar: Hyperparameter Tuning

Misalkan model bergantung pada vektor hyperparameter

$$\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_d) \in \Lambda,$$

dan fungsi skor validasi rata-rata (mis. akurasi atau F1) adalah

$$\hat{J}(\boldsymbol{\lambda}) = \frac{1}{K} \sum_{k=1}^K s^{(k)}(\boldsymbol{\lambda}),$$

dengan $s^{(k)}$ adalah skor pada lipatan (*fold*) ke- k .

Tujuan tuning:

$$\boldsymbol{\lambda}^* \in \arg \max_{\boldsymbol{\lambda} \in \Lambda} \hat{J}(\boldsymbol{\lambda}).$$

1.1 Grid Search

Ide. Menyapu kisi (*grid*) semua kombinasi kandidat setiap hyperparameter.

Keunggulan.

- Implementasi sangat sederhana.
- Reproducible: ruang yang dievaluasi deterministik.

Keterbatasan.

- **Eksplasi kombinasi** saat dimensi meningkat; biaya komputasi cepat membesar.
- Distribusi evaluasi tidak adaptif terhadap sensitivitas tiap dimensi.

Kompleksitas. Jika tiap dimensi memiliki m_i kandidat, total evaluasi

$$M_{\text{grid}} = \prod_{i=1}^d m_i.$$

1.2 Random Search

Ide. Mengambil sampel acak $\lambda^{(1)}, \dots, \lambda^{(N)}$ dari distribusi prior pada ruang hyperparameter.

Keunggulan.

- **Efisien di dimensi tinggi:** menutup kemungkinan pemborosan di dimensi yang tidak penting.
- Probabilistik: peluang menjangkau daerah baik meningkat dengan jumlah sampel.

Intuisi probabilistik. Misalkan proporsi volume ruang yang “bagus” adalah $p \in (0, 1]$. Probabilitas *gagal* menemukan daerah bagus setelah N sampel acak adalah

$$P(\text{gagal}) = (1 - p)^N,$$

sehingga *berhasil* minimal sekali adalah

$$P(\text{berhasil}) = 1 - (1 - p)^N.$$

Saat N naik, peluang menemukan setelan baik meningkat cepat bahkan untuk p kecil.

Keterbatasan.

- Hasil bervariasi antar-ulang; butuh *seed* untuk reproduksibilitas.
- Tanpa pengetahuan awal, distribusi sampel bisa kurang terarah (namun dapat diperbaiki dengan *log-uniform* untuk skala lebar).

1.3 Pemilihan Ruang Pencarian

- **Skala log** untuk parameter positif yang rentangnya lebar, mis. $C \in [10^{-3}, 10^3]$:

$$\log_{10} C \sim \mathcal{U}(-3, 3).$$

- **Distribusi diskret** untuk integer seperti jumlah tetangga $k \in \{1, 3, 5, 7, \dots\}$.
- **Kategori** untuk pilihan kernel, fungsi aktivasi, atau strategi penyeimbang kelas.

2 K-Fold Cross-Validation (CV)

2.1 Definisi

Bagi data menjadi K lipatan berukuran hampir sama: $\mathcal{D} = \cup_{k=1}^K \mathcal{D}_k$ dan saling lepas. Untuk setiap k :

$$\mathcal{D}_{\text{train}}^{(k)} = \mathcal{D} \setminus \mathcal{D}_k, \quad \mathcal{D}_{\text{val}}^{(k)} = \mathcal{D}_k.$$

Latih model pada $\mathcal{D}_{\text{train}}^{(k)}$, evaluasi pada $\mathcal{D}_{\text{val}}^{(k)}$ menghasilkan skor $s^{(k)}$. Estimator performa:

$$\hat{\mu} = \frac{1}{K} \sum_{k=1}^K s^{(k)}, \quad \hat{\sigma} = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (s^{(k)} - \hat{\mu})^2}.$$

2.2 Stratified K-Fold

Untuk klasifikasi, menjaga proporsi kelas serupa di tiap lipatan mengurangi variansi estimator:

Proporsi kelas di $\mathcal{D}_k \approx$ proporsi global.

2.3 Time-Aware (Time Series CV)

Jika data berurutan waktu, **jangan acak**:

$$\begin{aligned}\mathcal{D}_{\text{train}}^{(1)} &= [1 : T_1], & \mathcal{D}_{\text{val}}^{(1)} &= (T_1 : T_2], \\ \mathcal{D}_{\text{train}}^{(2)} &= [1 : T_2], & \mathcal{D}_{\text{val}}^{(2)} &= (T_2 : T_3], \text{ dst.}\end{aligned}$$

Meniru situasi prediksi masa depan, mencegah *leakage* informasi dari masa depan ke masa lalu.

2.4 Ukuran K dan Dampaknya

- K kecil (mis. 3–5): latih lebih cepat, variansi estimator lebih tinggi.
- K lebih besar (mis. 10): latih lebih lama, variansi lebih rendah.
- *Leave-One-Out* (LOOCV, $K = n$): bias rendah, variansi tinggi, mahal komputasi.

2.5 Nested CV (Singkat)

Untuk menghindari *optimism bias* pada tuning, gunakan CV *luar* untuk mengukur performa akhir dan CV *dalam* untuk memilih hyperparameter. Estimator akhir:

$$\hat{J}_{\text{nested}} = \frac{1}{K_{\text{outer}}} \sum_{o=1}^{K_{\text{outer}}} \max_{\lambda \in \Lambda} \frac{1}{K_{\text{inner}}} \sum_{i=1}^{K_{\text{inner}}} s^{(o,i)}(\lambda).$$

3 Praktik Anti-Leakage dalam CV

- **Semua** transformasi berbasis data (*imputation, scaling, encoding, feature selection*) harus dipelajari *hanya* dari $\mathcal{D}_{\text{train}}^{(k)}$ dan kemudian diaplikasikan ke $\mathcal{D}_{\text{val}}^{(k)}$.
- Gunakan **pipeline** agar urutan transformasi konsisten dan aman.
- Pada time series, pastikan **cutoff waktu** benar dan tidak mencampur masa depan ke masa lalu.

4 Grid vs Random: Rangkuman Praktis

Aspek	Grid Search	Random Search
Sederhana	Sangat mudah	Mudah
Skalabilitas dimensi	Buruk (eksplosi kombinasi)	Baik
Eksplorasi awal	Kurang fleksibel	Sangat cocok
Reproducible	Ya (deterministik)	Ya (dengan seed)
Menemukan interaksi halus	Baik pada ruang kecil	Baik dengan N memadai

5 Tracking Hasil Eksperimen

5.1 Mengapa Perlu Tracking?

- Menjaga **reproduksibilitas**: tahu persis konfigurasi yang menghasilkan skor.
- Memungkinkan **perbandingan adil**: metrik, data split, versi paket.
- Memudahkan **audit dan kolaborasi**.

5.2 Apa yang Harus Dicatat?

- **Metadata:** waktu, *experiment_id*, *run_id*, *seed*, hostname.
- **Data:** sumber dataset, ukuran train/val/test, skema fitur, *hash* data (opsional).
- **Konfigurasi:** nama model, ruang hyperparameter, nilai terpilih.
- **Prosedur:** jenis CV (*K*, stratified/time-aware), *pipeline* transformasi.
- **Metrik:** mean $\hat{\mu}$, std $\hat{\sigma}$, confusion matrix, ROC-AUC, dsb.
- **Artefak:** model terserialisasi, *scaler/encoder*, grafik penting.
- **Versi:** commit git, versi paket Python, OS.

5.3 Format Pencatatan Sederhana

CSV / JSON Log. Satu baris per *run*:

run_id, time, model, space, method, *K*, seed, mean_score, std_score, best_params, notes

Contoh (JSON singkat):

```
1 {
2   "run_id": "2025-12-01_14-10-22",
3   "model": "LogisticRegression",
4   "search": "random",
5   "space": {"C": "loguniform[1e-3, 1e3]", "penalty": ["l2"]},
6   "cv": {"type": "stratified", "k": 5, "seed": 42},
7   "score": {"metric": "f1_macro", "mean": 0.842, "std": 0.011},
8   "best_params": {"C": 2.31, "penalty": "l2"},
9   "data": {"train": 12000, "val_total": 3000, "features": 512},
10  "env": {"python": "3.12", "sklearn": "1.5.2"},
11  "git": {"commit": "a1b2c3d"},
12  "notes": "ngram=(1,2), min_df=3"
13 }
```

Struktur Direktori Artefak.

```
1 experiments/
2   2025-12-01_14-10-22/           # run_id
3     config.json
4     metrics.json
5     model.pkl                   # model terlatih
6     pipeline.pkl                # full pipeline (preprocess +
7   model)                        # confusion matrix
8     cm.png                     # daftar versi paket
9     requirements-freeze.txt
```

5.4 Checklist Reprodusibilitas

- Tetapkan **random seed** untuk semua komponen yang relevan.
- Simpan **split** (indeks fold) atau simpan **generator** CV.
- **Freeze** versi paket pada saat *best run*.
- Simpan **pipeline lengkap** (anti-leakage) dan **konfigurasi**.

6 Contoh Pseudocode (sklearn)

6.1 Grid Search + Stratified K-Fold

```
1 from sklearn.model_selection import StratifiedKFold, GridSearchCV
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.linear_model import LogisticRegression
5
6 pipe = Pipeline([
7     ("sc", StandardScaler(with_mean=False)), # contoh bila fitur sparse:
8         with_mean=False
9     ("clf", LogisticRegression(max_iter=1000))
10 ])
11
12 param_grid = {
13     "clf__C": [0.01, 0.1, 1, 10, 100],
14     "clf__penalty": ["l2"],
15 }
16
17 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
18 search = GridSearchCV(pipe, param_grid, scoring="f1_macro", cv=cv,
19     n_jobs=-1)
20 search.fit(X, y)
21
22 best = search.best_params_
23 mean = search.best_score_
```

6.2 Random Search (log-uniform) + Tracking Ringkas

```
1 import numpy as np, json, time
2 from scipy.stats import loguniform
3 from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV
4
5 param_dist = {
6     "clf__C": loguniform(1e-3, 1e3),
7 }
8
9 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
10 rnd = RandomizedSearchCV(pipe, param_distributions=param_dist,
11     n_iter=40, scoring="f1_macro", cv=cv, n_jobs
12     =-1,
13     random_state=42)
14 rnd.fit(X, y)
15
16 run_id = time.strftime("%Y-%m-%d_%H-%M-%S")
17 log = {
18     "run_id": run_id,
19     "search": "random",
20     "best_params": rnd.best_params_,
21     "best_score": float(rnd.best_score_)
22 }
23
24 with open(f"experiments/{run_id}_metrics.json", "w") as f:
25     json.dump(log, f, indent=2)
```

7 Strategi Praktis

- Mulai dengan **random search** untuk eksplorasi cepat; lanjutkan **grid lokal** di sekitar rentang menjanjikan.
- Gunakan **skala log** untuk hyperparameter sensitif skala (mis. C , learning rate).
- Pastikan **CV sesuai data**: stratified untuk klasifikasi; time-aware untuk data berurut.
- Terapkan **pipeline** agar aman dari *leakage* dan mudah diserialisasi.
- **Track** semua run penting; simpan artefak agar mudah *roll-back*.

8 Ringkasan

- **Grid** cocok untuk ruang kecil dan kebutuhan deterministik.
- **Random** efektif di ruang besar/tinggi dimensi; peluang temuan baik meningkat dengan jumlah sampel.
- **K-Fold** memberikan estimasi performa yang lebih stabil dibanding satu split; pilih variasi yang sesuai (stratified / time-aware).
- **Tracking hasil** adalah fondasi reproduisibilitas dan kolaborasi yang sehat.